

# NoSQL Models and JSON

---

A ticket can have a requester,  
a changing status, and years of events.

Day 1: the data models behind different application needs

Day 2: two JSON designs for the same ticket

# Relational ideas we will keep

---

| Fact                     | Relational representation            | Why it matters                               |
|--------------------------|--------------------------------------|----------------------------------------------|
| A person has an identity | users.user_id is a primary key       | Names can change without changing identity   |
| A ticket has a requester | tickets.requester_id refers to users | A relationship should have a defined meaning |
| A ticket has many events | Each event refers to its ticket      | Separate history can grow independently      |

A document design still needs identity, relationships, and rules.  
The main change is where related facts live.

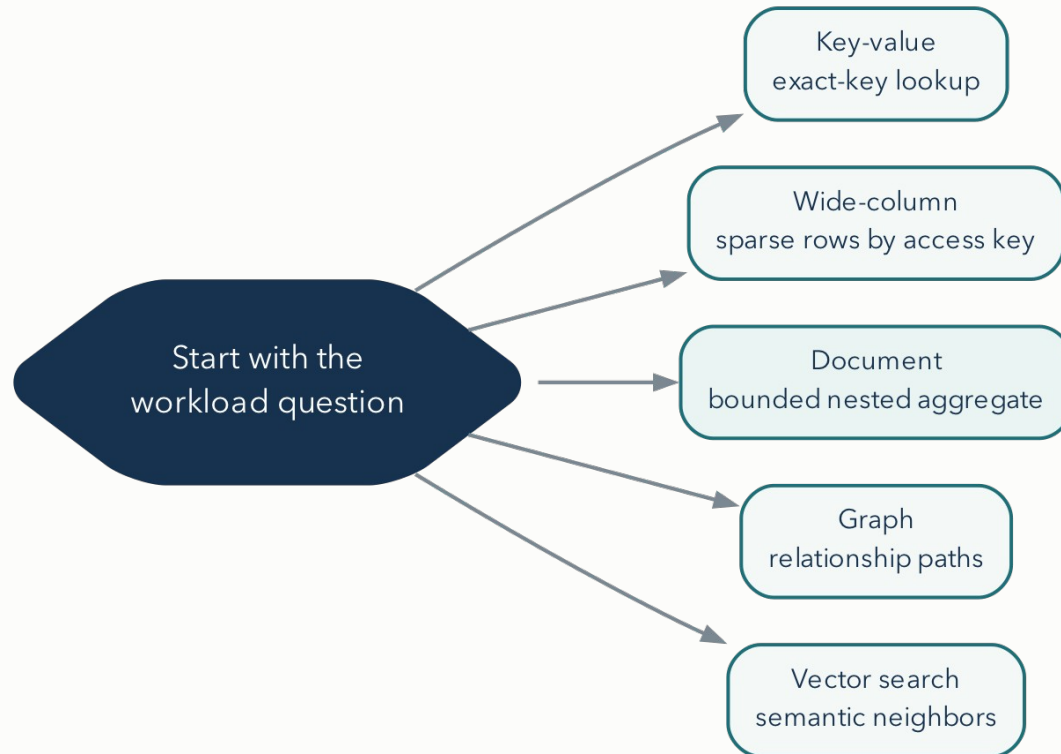
# Historical roots and distributed workloads

---

| Anchor               | Contribution                                                |
|----------------------|-------------------------------------------------------------|
| 1970: Codd           | Relations and data independence                             |
| Earlier NoSQL name   | Strozzi used the name for a relational system without SQL   |
| 2006: Bigtable       | A sparse distributed map for large structured datasets      |
| 2007: Dynamo         | Key-value storage designed around availability requirements |
| Later NoSQL movement | A broad label for several nonrelational approaches          |

These designs address different needs. Dynamo is the research system, not the same product as today's DynamoDB service.

# Model families and access patterns



Access pattern: the data an application repeatedly reads or changes.

Opening ticket 1001 and its latest events is one pattern.

Counting every event by type is another.

One application may benefit from more than one representation.

# Key-value lookup

---

**session:8f21**



**Stored session value**

```
user_id: 101  
expires_at: 2026-10-20T19:30:00Z
```

Known key: fetch this session.

Different request: find every session belonging to user 101.

The second request may need another index or mapping.

The value shown is conceptual notation; it is not a command or JSON.

# Wide-column rows and key design

| Illustrative row key       | Observed fields                |
|----------------------------|--------------------------------|
| device07#2026-10-20T09:00Z | temperature: 21.4              |
| device07#2026-10-20T09:01Z | temperature: 21.6, battery: 78 |
| device12#2026-10-20T09:00Z | humidity: 61                   |

A device-prefix range keeps one device's readings together.  
A cross-device battery report has a different access pattern.

Bigtable-style illustration. Wide-column storage differs from analytical columnar storage.

# Documents and bounded related data

---

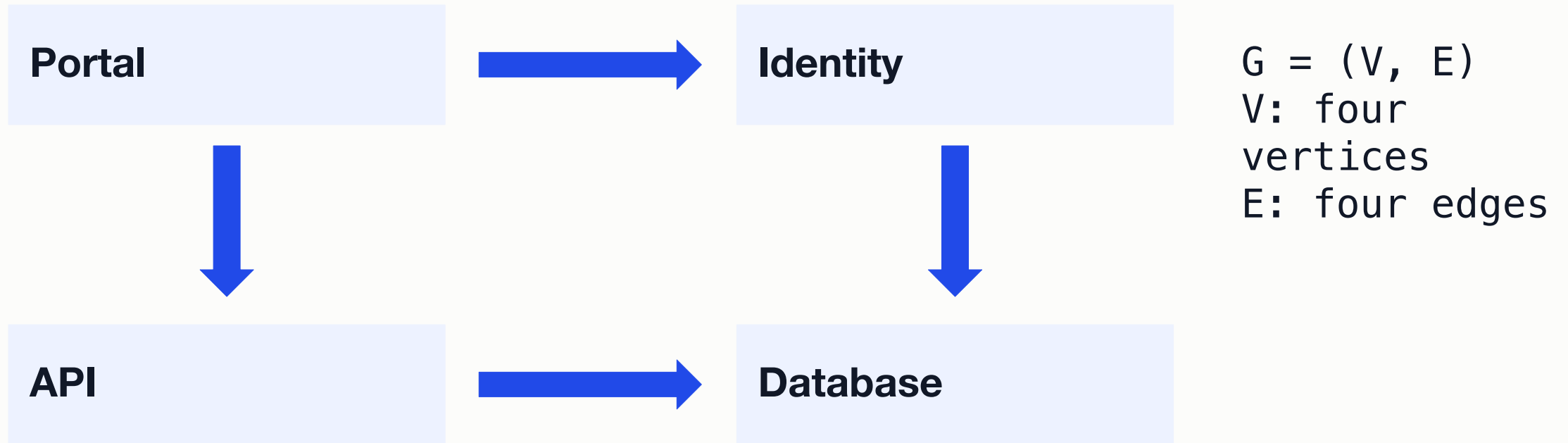
```
{  
  "ticket_id": 1001,  
  "status": "open",  
  "requester": {"user_id": 101, "display_name": "Maya Chen"},  
  "tags": ["streetlight", "safety"]  
}
```

One document contains the ticket and selected related details.  
The nested requester is still a relationship.

Shape preview only. Day 2 teaches the punctuation.  
The tags illustrate an added field rather than a column copied from the CSV.

# Directed graphs: vertices and edges

---



Each arrow means “depends on.” Portal has two outgoing edges. Portal can reach Database through Identity or API.

# Reachability and breadth-first search

---

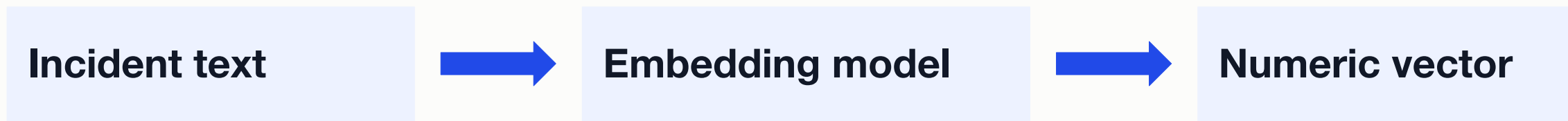
| Traversal from Portal | Newly reached vertices |
|-----------------------|------------------------|
| 0 edges               | Portal                 |
| 1 edge                | Identity, API          |
| 2 edges               | Database               |

Breadth-first search visits one edge-distance level at a time.  
A visited set prevents repeated visits, including in cycles.

Fewest edges does not mean least latency when edges have different weights.  
A dependency path alone does not prove that an outage will propagate.

# Vectors and similar incident descriptions

---



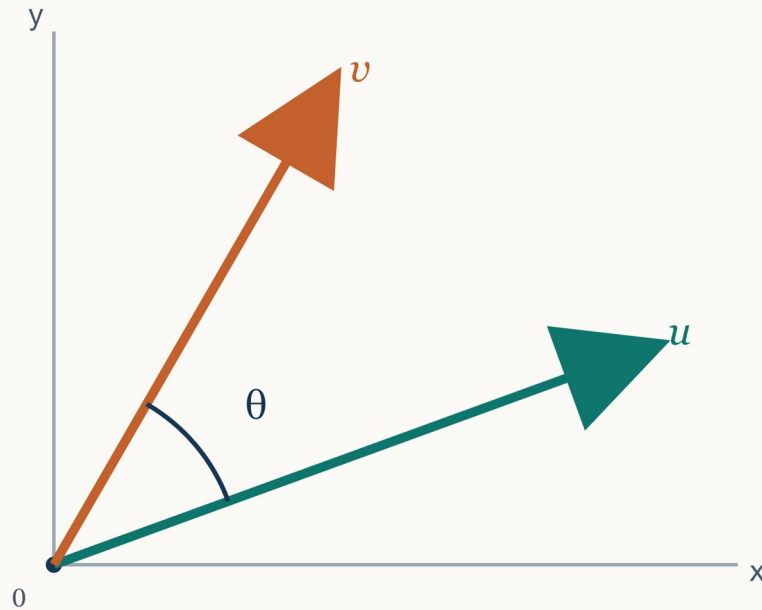
“Lamp outside the library is flickering.”  
“Streetlight repeatedly turns off and on.”

A model can place related descriptions near each other.  
The database stores the vectors and retrieves nearby candidates.

No embedding API key or vector deployment is required this week.

# Vector direction and length

The angle between two nonzero vectors



$$\cos \theta = (u \cdot v) / (\|u\| \|v\|)$$

Same direction: cosine = 1

Perpendicular: cosine = 0

Opposite direction: cosine = -1

$\|u\|$  and  $\|v\|$  are Euclidean lengths.

Theta is the angle between  $u$  and  $v$ . Scaling a vector by a positive factor changes its length but preserves its direction.

## Cosine similarity: a complete calculation

$$q = (1, 0) \quad a = (10, 0) \quad b = (1, 1)$$

$$q \cdot a = 1 \times 10 + 0 \times 0 = 10$$

$$q \cdot b = 1 \times 1 + 0 \times 1 = 1$$

Lengths:  $\|q\| = 1$ ,  $\|a\| = 10$ ,  $\|b\| = \sqrt{2}$

$$\cos(q, a) = 10 / (1 \times 10) = 1$$

$$\cos(q, b) = 1 / (1 \times \sqrt{2}) \approx 0.707$$

Cosine ranks a first. A zero-length vector would make the denominator zero.

## Euclidean distance reverses this ranking

---

Distance =  $\sqrt{(\text{sum of squared coordinate differences})}$

$$\begin{aligned}d(q, a) &= \sqrt{(1 - 10)^2 + (0 - 0)^2} = 9 \\d(q, b) &= \sqrt{(1 - 1)^2 + (0 - 1)^2} = 1\end{aligned}$$

Euclidean distance ranks b first.

Cosine ignores the positive change in length that makes a farther away.

For unit-length vectors:  $\text{distance}^2 = 2 - 2 \times \text{cosine}$ .

On the same normalized vectors, the exact rankings agree, allowing for ties.

# Approximate search and useful results

---

Exact top five IDs: A, B, C, D, E

Approximate top five IDs: A, C, E, F, G

Recall at five =  $3 / 5 = 0.6$

Recall measures agreement with the exact retrieval target.  
People still have to judge whether the results help with the incident.

Similarity does not grant permission to read a private ticket.  
Embedding choice, index settings, and access controls all affect the system.

# Model choice for a stated workload

| Workload                          | Candidate     | A cost to investigate                |
|-----------------------------------|---------------|--------------------------------------|
| Known session token               | Key-value     | Finding sessions without their keys  |
| One device's ordered readings     | Wide-column   | Other query patterns and hot keys    |
| One ticket with bounded details   | Document      | Shared facts and growing arrays      |
| Multi-hop dependencies            | Graph         | Traversal size and edge meaning      |
| Similar incident descriptions     | Vector search | Retrieval quality and model versions |
| Shared facts and changing reports | Relational    | Joins and schema evolution           |

These are starting points. Products often support more than one model.

## A short design exercise

---

The support portal needs a ticket page and a report of event counts.

Choose a useful representation for the ticket page.

Then explain one consequence for the report or for changing a name.

Work individually in your notes. No separate submission.

Keep the relational option available and justify any additional store.

## Day 2: JSON and the ticket page

---

The same facts can appear  
in different document shapes.

Syntax and types, followed by two worked designs  
GitHub editor and Atlas orientation  
Individual CSV-to-JSON lab

# JSON as a language-independent format

---

2006: RFC 4627 documents JSON

2014: RFC 7159 revises interoperability rules

2017: RFC 8259 becomes the Internet standard

```
{"ticket_id": 1001, "status": "open"}
```

JavaScript, Python, and other languages can exchange the same text. Objects and arrays directly express nested API responses.

JSON describes data. It does not execute a query or provide a database.

# JSON has six value kinds

| Kind    | Example                                 | Meaning                        |
|---------|-----------------------------------------|--------------------------------|
| Object  | <code>{"status": "open"}</code>         | Named members                  |
| Array   | <code>["streetlight", "safety"]</code>  | Ordered values                 |
| String  | <code>"open"</code>                     | Quoted text                    |
| Number  | <code>1001</code>                       | Numeric value                  |
| Boolean | <code>true</code> or <code>false</code> | Two literals in one value kind |
| Null    | <code>null</code>                       | An explicit null value         |

Any JSON value can be the top level. Our lab deliberately uses an object.

# Objects, arrays, and punctuation

---

```
{  
  "ticket_id": 1004,  
  "subject": "Broken bench slat",  
  "status": "new",  
  "assignee_id": null,  
  "tags": ["parks", "bench"]  
}
```

{ } enclose an object

[ ] enclose an array

: connects name and value

, separates neighbors

Names and strings use double quotes. Indentation shows containment.  
The tags are an illustrative addition to ticket 1004.

# A syntax repair

---

```
{'status': 'open',}
```

Two errors: single quotes and a comma after the final member.

```
{"status": "open"}
```

Comments, NaN, Infinity, and the number 01 are also outside standard JSON. A permissive parser may accept extensions. Acceptance alone is insufficient.

# Missing, null, empty, and zero

| JSON fragment                      | What the text supplies    |
|------------------------------------|---------------------------|
| <code>{}</code>                    | No assignee property      |
| <code>{"assignee_id": null}</code> | An explicit null assignee |
| <code>{"events": []}</code>        | An empty array            |
| <code>{"estimated_cost": 0}</code> | A numeric zero            |

The application decides whether an omitted field and null mean the same thing. An empty event array does not explain why history is absent.

# JSON text and MongoDB BSON

---

```
{"opened_at": "2026-02-01T23:10:00Z"}
```

This value is a JSON string with an agreed timestamp format.

BSON supports additional storage types, including Date and ObjectId. A driver or an import step decides how to create those stored types.

Use unique member names. Keep IDs as strings when their exact spelling matters. A valid document can still violate the application's data contract.

## The worked example uses ticket 1001

---

| CSV               | Relevant records                                                                      |
|-------------------|---------------------------------------------------------------------------------------|
| users.csv         | 101: Maya Chen. 201: Priya Shah.                                                      |
| tickets.csv       | 1001: Streetlight dark near bus stop. Status: open.<br>Requester: 101. Assignee: 201. |
| ticket_events.csv | 5001: created, 2026-02-01T23:10:00Z<br>5002: assigned, 2026-02-02T14:05:00Z           |

Both upcoming examples preserve these facts. Other CSV columns are omitted. The student lab uses a different ticket, 1003, with three events.

## Referenced design: IDs connect separate records

```
{
  "users": [
    {"user_id": 101, "display_name": "Maya Chen"},
    {"user_id": 201, "display_name": "Priya Shah"}
  ],
  "tickets": [{
    "ticket_id": 1001, "subject": "Streetlight dark near bus stop",
    "status": "open", "requester_id": 101, "assignee_id": 201
  }],
  "events": [
    {"event_id": 5001, "ticket_id": 1001, "event_type": "created",
     "event_at": "2026-02-01T23:10:00Z"},
    {"event_id": 5002, "ticket_id": 1001, "event_type": "assigned",
     "event_at": "2026-02-02T14:05:00Z"}
  ]
}
```

One JSON object packages three arrays. This file is not yet a MongoDB database.

## Embedded design: related facts inside the ticket

---

```
{
  "ticket_id": 1001,
  "subject": "Streetlight dark near bus stop",
  "status": "open",
  "requester": {"user_id": 101, "display_name": "Maya Chen"},
  "assignee": {"user_id": 201, "display_name": "Priya Shah"},
  "events": [
    {"event_id": 5001, "event_type": "created",
     "event_at": "2026-02-01T23:10:00Z"},
    {"event_id": 5002, "event_type": "assigned",
     "event_at": "2026-02-02T14:05:00Z"}
  ]
}
```

The containing ticket identifies the events' owner.  
The document carries copies of the two names.

## A current name and a historical name

---

| Meaning of the copied name    | Consequence when Maya changes it                                            |
|-------------------------------|-----------------------------------------------------------------------------|
| Current display name          | Every current-name copy must eventually match the authoritative user record |
| Name when the ticket opened   | Keep the old name deliberately and label it as a snapshot                   |
| User ID without a copied name | Look up the authoritative user record for the current name                  |

A stale current name is an inconsistency.

A documented historical snapshot preserves a different fact.

# Bounded recent events and complete history

---

## Ticket document

Ticket ID and status

At most five recent events

Fast ticket-page preview

## Separate event records

ticket\_id links each event

All retained events

Independent history and reports

A copied recent-event list needs a refresh rule.

This design does not make changes across separate documents automatically atomic.

# GitHub's browser text editor

---

Your repository: Add file > Create new file  
Filename: week\_09\_json\_models.md

```
## Referenced design
```json
{"ticket_id": 1004, "assignee_id": null}
```
```

My explanation goes below the JSON block.

Preview checks the Markdown display. Commit changes saves the text.  
Download the file for Brightspace. Neither step validates the JSON syntax.

# Atlas orientation and connection boundaries

---

| Component                 | Purpose                                            |
|---------------------------|----------------------------------------------------|
| Atlas account and project | Manage the cloud deployment                        |
| Free cluster              | Run the database service with free-tier limits     |
| Database user             | Authenticate a database connection                 |
| IP access list            | Permit a source network to attempt that connection |

Choose Free / M0 and only the access needed for your practice.  
Do not enter a payment card or publish credentials.

Today's JSON lab needs no cloud connection. Week 10 introduces MQL.

## Lab: one ticket, two JSON designs

---

Ticket 1003. Requester 103. Assignee 201.  
Events 5005, 5006, and 5007 from the Metro Support CSVs.

Write one referenced design and one embedded or hybrid design.

Explain how the ticket page reads the facts and how name changes and years of history affect your choice.

Individual work in a text editor. No SQL, MQL, or Python required.  
Brightspace: one Markdown file containing both examples and the paragraph.

## Explaining a document-design decision

---

I kept user IDs so a name change would not change identity. For the ticket page, I considered embedding a small set of recent events. I would keep the complete history separately because it continues to grow. A recent-event copy would need a refresh rule so readers could distinguish an up-to-date preview from an older one.

The lab practices this reasoning with concrete JSON.  
Next week: retrieve and change documents with MQL.